Bash on WSL2 for the Data & ML Engineer

A from-zero tutorial for Python developers moving onto Linux through Windows

Abstract

This tutorial takes you from *nothing installed* to *daily-driver bash on Linux* in one document, assuming only that you know Python. You'll install WSL2 and Ubuntu, learn the bash mental model, work with files, pipes, streams, CSV, JSON, and HTTP, manage Python with `uv` and `venv`, run Docker containers, get a GPU visible from Linux for PyTorch, and write your first ETL script. Every section uses realistic data and a Python comparison where it helps. If you have read the companion PowerShell tutorial, occasional blue callouts cross-reference the two – but PowerShell knowledge is not required.

Contents

Part I — WSL2 Setup and Mental Model

1 What WSL2 Is, and Why Bash for Data Work

WSL2 – the *Windows Subsystem for Linux, version 2* – is a real Linux kernel that Microsoft ships as a built-in Windows feature. It is not an emulator and not a virtual machine in the heavy sense: it boots in under a second, shares your CPU and memory with Windows, and lets you access your Windows drives at `/mnt/c/`, `/mnt/d/`, etc. Inside it, you get a complete Ubuntu (or Debian, or Fedora…) userland that behaves identically to a Linux server.

For a data or ML engineer, the case for living in WSL2 is simple:

- Every production system you'll touch runs Linux: EC2 instances, Docker containers, Kubernetes pods, CI runners, ML training rigs.
- Every modern data tool (`uv`, `docker`, `aws`, `kubectl`, `terraform`, `dbt`, `spark-submit`, `mongosh`, `mysql`) is built bash-first.
- Tutorials, Stack Overflow answers, and your future colleagues assume bash. Skills transfer to every job; PowerShell skills mostly do not.
- Python on WSL2 is smoother than Python on native Windows – no path-separator surprises, no `\r\n` line-ending bugs, native CUDA extensions install cleanly.

The bash mental model in one paragraph

Every bash command reads bytes from *standard input* (stdin) and writes bytes to *standard output* (stdout) and *standard error* (stderr). The pipe operator `|` connects one command's stdout to the next command's stdin. Redirection (`>`, `>>`, `<`) connects those streams to files. Because everything is bytes, you compose programs out of small text-processing tools: `grep` finds lines, `awk` slices columns, `sed` edits substrings, `jq` navigates JSON, `sort` and `uniq` aggregate. Master this composition pattern and you can solve any data-shaping problem on the command line.

If you read the PowerShell tutorial

PowerShell pipes *objects* (records with named properties); bash pipes *bytes*. Both work, and PowerShell's model is arguably more elegant for structured data – but every Linux server in the world runs bash, so the byte pipeline is what pays the bills. Tools like `jq` (for JSON) and `mlr` (for CSV) recover most of PowerShell's structured-pipeline ergonomics inside bash.

2 Installing WSL2 and Ubuntu

The install is one command. From a normal (non-admin) Windows PowerShell or Command Prompt:

```
wsl --install -d Ubuntu
```

This downloads the WSL2 kernel, installs Ubuntu (the most widely-supported distro for data work), and reboots once. After reboot, Ubuntu boots and asks for a Linux username and password. Use something short – you'll type the username constantly – and pick a password you can remember; you'll need it for `sudo` (privileged commands).

After first login, update the package index and install a small starter kit:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y \
    build-essential git curl wget unzip jq htop tree \
    ca-certificates gnupg lsb-release
```

What each tool gives you:

Package	Why
`build-essential`	C/C++ compilers; needed by some pip packages
`git`	Source control
`curl` / `wget`	Fetch URLs (curl is the modern default)
`unzip`	Unzip archives
`jq`	Command-line JSON processor (essential)
`htop`	Interactive process viewer (better `top`)
`tree`	Pretty directory tree

Windows Terminal as your shell host

Open **Windows Terminal** (preinstalled on Windows 11; otherwise install from the Microsoft Store). Click the dropdown next to the tab bar and pick `Ubuntu`. You can pin Ubuntu as the default profile in Settings → Startup. From now on, every time you want a Linux shell, you open Windows Terminal and you're there.

```
# Confirm you're in Ubuntu:
cat /etc/os-release
# NAME="Ubuntu"
# VERSION="24.04.x LTS (Noble Numbat)"
# ...

uname -a              # kernel info
whoami                # your linux username
```

The `sudo` command

You'll see `sudo` in front of many commands. It runs the next command as root (the Linux superuser). The first `sudo` in a session asks for your password; subsequent ones within a few minutes don't. Use it only when you need to: installing system packages, editing files in `/etc/`, managing services. **Never** use `sudo pip install` – that pollutes the system Python.

3 The Linux Filesystem and Where Your Code Lives

The single most important WSL2 lesson: **keep your code under `~/projects/`, not `/mnt/c/`.** The `/mnt/c/` mount lets you see your Windows `C:` drive from Linux, which is convenient – but it goes through a translation layer that is roughly 10x slower for everything git, pip, or `node_modules` touches. Code in your Linux home directory lives on the native Linux filesystem and is fast.

```
echo $HOME              # /home/shiru
cd ~                    # ~ is shorthand for $HOME
mkdir -p ~/projects     # this is where every project goes
cd ~/projects
```

Linux filesystem orientation

Path	What lives there
`/`	The root of the filesystem (everything below)
`/home/shiru`	Your home (also written `~`); your projects, config, dotfiles
`/etc`	System configuration (DNS, apt sources, hosts, ssh)
`/var`	Variable data: logs, package caches, mail
`/tmp`	Temporary files; cleared at reboot
`/usr/bin`	Most installed programs
`/usr/local/bin`	Programs you installed manually (precedes `/usr/bin` on PATH)
`/opt`	Optional/third-party packages (e.g. CUDA toolkit)
`/mnt/c`	Your Windows C: drive (slow; avoid for code)

Accessing your Linux files from Windows

Open Windows Explorer and type `\\wsl$\Ubuntu\home\shiru` in the address bar – you can browse your Linux home like any Windows folder. Drag-and-drop works in both directions. The reverse also works: from Linux, your Windows desktop is at `/mnt/c/Users/shiru/Desktop`.

VS Code with the WSL extension

Install the **Remote – WSL** extension in VS Code (on the Windows side). Then, from inside WSL, run:

```
cd ~/projects/my-app
code .
```

VS Code launches on Windows, but the editor, terminal, debugger, and language servers all run inside Ubuntu. Your venvs activate correctly; your file paths are POSIX; everything Just Works.

Gotcha

If you ever see file paths like `C:\Users\shiru\projects` inside a bash prompt, you've accidentally opened a Windows folder from `/mnt/c/`. **Close it and reopen from `~/projects/`.** You will lose hours to slow git operations otherwise.

Part II — Bash Fundamentals

4 Getting Around: Files and Directories

The dozen commands you'll use every day:

```
pwd                       # print working directory
cd ~/projects             # change directory
cd -                      # cd back to the previous directory
ls                        # list files
ls -la                    # long format, including hidden (dot-files)
ls -lh                    # human-readable sizes (KB, MB, GB)
ls -lt                    # sort by modification time, newest first

mkdir data                # create one dir
mkdir -p data/raw/2026    # create nested dirs (-p = "make parents")

touch notes.txt           # create empty file (or update its mtime)
cp users.csv backup.csv   # copy
cp -r src/ src_backup/    # copy directory recursively
mv old.csv new.csv        # rename or move
rm tmp.csv                # delete a file
rm -rf .venv/             # delete a directory and everything in it (DANGEROUS)

ln -s ~/data/2026 ./data  # symbolic link (shortcut)
which python3             # show which python3 is on PATH
realpath ./users.csv      # absolute path, with symlinks resolved

cat /etc/os-release       # print a file to stdout
file data.bin             # what KIND of file is this?
stat users.csv            # size, mtime, permissions, inode
du -sh ~/.cache           # disk usage of a directory (s = summary, h = human)
df -h                     # disk space free, per mounted filesystem
```

Globbing (filename patterns)

The shell expands globs into a list of matching filenames *before* the command runs. So `ls *.csv` runs as `ls users.csv orders.csv ...` after expansion.

```
ls *.csv                  # every .csv in this directory
ls data/*.csv             # every .csv directly under data/
shopt -s globstar          # enable ** (one-time per shell; put it in ~/.bashrc)
ls **/*.parquet           # every .parquet at any depth
ls report_2026-??.csv     # ? matches exactly one char: 2026-01..2026-99
ls report_2026-[01][0-9].csv  # only months 00-19
```

Tab completion is the killer feature

Hit `Tab` after typing the first few letters of any command, filename, or argument and bash completes it (or shows choices if there are several). `Tab` also works on program-specific arguments after you install `bash-completion` – e.g. `git ch<Tab>` expands to `git checkout`.

Gotcha

`rm -rf` with a typo in the path is unrecoverable – there is no Recycle Bin in bash. Get in the habit of running `ls <target>` first to confirm what `rm` would see, and use absolute paths (`rm -rf /home/shiru/.venv`) when in doubt.

Permissions in 60 seconds

Every file has three permission triples – owner, group, other – each with read/write/execute bits. `ls -l` prints them:

```
-rwxr-xr--  1  shiru  shiru  4096  May 25 11:00  script.sh
# Owner: rwx (read, write, execute)
# Group: r-x (read, execute)
# Other: r-- (read only)

chmod +x script.sh        # make executable for everyone
chmod 644 notes.txt       # 6=rw-, 4=r--, 4=r--   (the standard "file" mode)
chmod 755 bin/run.sh      # 7=rwx, 5=r-x, 5=r-x   (the standard "script" mode)
chown shiru:shiru file    # change owner/group (usually with sudo)
```

For everyday work: scripts need `chmod +x`; everything else keeps the default `644`.

5 Pipes, Streams, and Redirection

This is the section that pays for the whole tutorial. The four fundamental ideas:

1. Every command reads from `stdin` (file descriptor 0) and writes to `stdout` (1) and `stderr` (2).
2. `|` connects one command's stdout to the next command's stdin.
3. `>`, `>>`, and `<` connect those streams to files.
4. Commands return an exit code: `0` = success, anything else = failure.

The pipe

```
# How many lines in app.log contain "ERROR"?
cat app.log | grep ERROR | wc -l

# Equivalent and slightly better (grep can read the file directly):
grep ERROR app.log | wc -l

# Even better -- grep counts with -c:
grep -c ERROR app.log
```

Redirection: writing streams to files

```
# Overwrite a file with command output:
ls -la > listing.txt

# Append (don't overwrite):
date >> log.txt

# Read stdin from a file:
sort < unsorted.txt

# Send stderr to a file:
make 2> errors.log

# Send stdout and stderr to the same file:
make > build.log 2>&1
# or, in bash specifically:
make &> build.log

# Discard stderr entirely:
find / -name "*.log" 2>/dev/null

# Tee -- write to a file AND continue piping:
ls -la | tee listing.txt | wc -l
```

Chaining with `&&` and `||`

```
# Run cmd2 only if cmd1 succeeded:
mkdir -p dist && cp build/* dist/

# Run cmd2 only if cmd1 failed:
ping -c 1 example.com || echo "network is down"

# Sequence (always run both, regardless of result):
make clean ; make build
```

Exit codes and `$?`

```
grep foo missing.txt
echo $?                    # 2 -- file not found
grep foo present.txt
echo $?                    # 0 -- match found
grep nothing present.txt
echo $?                    # 1 -- no match
```

Exit codes are the bash equivalent of return values for control flow. Scripts use them in `if`

conditions; pipelines use `set -o pipefail` (Section 17) to make them propagate.

Compare with Python

In Python you compose functions; in bash you compose programs by piping streams. The shape of a 3-stage pandas method chain (`df.dropna().query('x > 0').to_csv(out)`) maps directly to a 3-stage shell pipeline. The difference is that bash pipes are text; you'll often shape the text into columns or JSON (`awk`, `jq`) before further processing.

6 Variables, Quoting, and Substitution

Variables

```
name="shiru"              # NO spaces around = (parse error otherwise)
count=42

echo $name                # shiru
echo "$name"              # shiru  (the safe form)
echo ${name}              # shiru  (braces let you concatenate)
echo "${name}_v2"         # shiru_v2  (without braces, bash would look for
    $name_v2)
```

Gotcha

`FOO = bar` (with spaces) is not an assignment – bash sees `FOO` as a command and tries to run it. Always `FOO=bar`, no spaces.

Quoting: the one rule you must internalize

- **Double quotes** interpolate `$VAR` and `$(cmd)` but preserve everything else literally.
- **Single quotes** preserve *everything* literally – no interpolation. Use single quotes for regex patterns, jq queries, SQL strings with `$`.
- **No quotes** causes word-splitting on whitespace and glob expansion. Almost always wrong for variables – always `"$var"` unless you specifically want splitting.

```
greeting="hello world"
echo "$greeting"          # hello world             (one argument)
echo $greeting            # hello world             (TWO arguments to echo!)
echo '$greeting'          # $greeting               (literal)

# A file with a space in its name:
ls "my file.txt"          # works
ls my file.txt            # tries to ls TWO files: "my" and "file.txt"
```

Parameter expansion

The single most useful family of bash features. `${VAR}` extends to a rich mini-language:

```
name="users.csv"

${name:-default.csv}       # 'users.csv' if set, else 'default.csv'
${UNSET:-default.csv}      # 'default.csv'

${name#*.}                  # 'csv'       -- strip shortest prefix matching *.
${name%.*}                  # 'users'     -- strip shortest suffix matching .*
${name##*.}                 # 'csv'       -- strip longest prefix matching *.
${name%%.*}                 # 'users'     -- strip longest suffix matching .*

filepath="/home/shiru/data/users.csv"
${filepath##*/}             # 'users.csv' -- basename
${filepath%/*}              # '/home/shiru/data' -- dirname

s="HELLO"
${s,,}                      # 'hello' -- lowercase (bash 4+)
${s^^}                      # 'HELLO' -- uppercase

arr=(a b c d e)
${arr[@]}                   # all elements
${#arr[@]}                  # 5 -- number of elements
${arr[2]}                   # 'c'
${arr[@]:1:2}               # 'b c' -- slice from index 1, length 2
```

Command substitution

```
today=$(date +%F)           # '2026-05-25'
echo "Backup file: backup-$today.tar.gz"

# Nested:
size=$(du -sh "$(pwd)" | cut -f1)
echo "This directory is $size"

# Arithmetic:
n=$((2 + 3 * 4))            # 14
echo $n
i=$((i + 1))                # increment

# Backticks `cmd` also work but are deprecated -- use $(cmd).
```

Arrays

```
fruits=(apple banana cherry)
fruits+=(date)              # append

echo ${fruits[0]}            # apple
echo ${fruits[@]}            # apple banana cherry date

# Iterate (note the quoting!):
```

```
for f in "${fruits[@]}"; do
    echo "got: $f"
done

# Read a file into an array, one line per element:
mapfile -t lines < users.csv
echo ${#lines[@]}            # number of lines
echo ${lines[0]}             # first line
```

Compare with Python

Bash arrays are like Python lists, but indexing is via `${arr[i]}` (not `arr[i]`), length is `${#arr[@]}` (not `len(arr)`), and – crucially – you must quote `"${arr[@]}"` when expanding, or elements with spaces split into multiple arguments. Bash also has *associative arrays* (`declare -A`), which are like Python dicts, but you'll reach for `jq` or Python long before you need them.

7 The Essential Text Toolkit

The five tools that turn bash into a data engineer's playground: `grep`, `sed`, `awk`, `sort`, `uniq`. Learn them in this order, in roughly this depth.

`grep` – find lines

```
grep ERROR app.log              # lines containing ERROR
grep -i error app.log           # case-insensitive
grep -v INFO app.log            # invert: lines NOT containing INFO
grep -c ERROR app.log           # count matching lines
grep -l ERROR *.log             # list FILENAMES with matches
grep -n ERROR app.log           # prefix each match with its line number
grep -r ERROR ~/projects         # recurse through a directory tree
grep -E '^(ERR|WARN)' app.log   # extended regex (alternation needs -E)
grep -A 3 -B 1 ERROR app.log    # 3 lines After, 1 line Before each match
```

`sed` – search and replace, line by line

```
sed 's/old/new/' file.txt              # replace first occurrence per line
sed 's/old/new/g' file.txt             # global: every occurrence
sed -i 's/old/new/g' file.txt          # in-place edit (-i)
sed -i.bak 's/old/new/g' file.txt      # in-place, but keep file.txt.bak

# Print only lines 10-20:
sed -n '10,20p' file.txt

# Delete lines matching a pattern:
sed '/^DEBUG/d' app.log

# Replace using regex backreferences:
sed -E 's/^([A-Z]+)-([0-9]+)$/\2,\1/' tickets.txt
```

Gotcha

`sed -i` on macOS requires `sed -i ''` (empty string). On Linux/WSL2 you use `sed -i` alone, or `sed -i.bak` to keep a backup. If a script needs to run on both, use `gsed` or just shell out to Python.

`awk` – the data engineer's secret weapon

`awk` treats each line as a record split into fields. The default field separator is whitespace; `-F,` sets it to a comma.

```
# Print the first column of every line:
awk '{print $1}' users.csv

# Filter rows where the 3rd CSV column > 100:
awk -F, '$3 > 100 {print $1, $3}' data.csv
```

```
# Sum the 2nd column:
awk -F, '{sum += $2} END {print sum}' sales.csv

# Average:
awk -F, 'NR>1 {sum += $3; n++} END {print sum/n}' users.csv
# NR is the current row number; NR>1 skips the header.

# Group by + count -- "how many rows per role?":
awk -F, 'NR>1 {n[$3]++} END {for (r in n) print r, n[r]}' users.csv
```

`sort` and `uniq`

```
sort file.txt                  # alphabetic sort
sort -n file.txt               # numeric sort
sort -r file.txt               # reverse
sort -k 2 -n -r file.txt       # sort by 2nd column, numerically, descending
sort -t, -k 3 -n -r data.csv   # CSV: -t, sets the separator

uniq file.txt                  # collapse adjacent duplicates
sort file.txt | uniq           # the canonical "unique values" idiom
sort file.txt | uniq -c        # count each unique value
sort file.txt | uniq -c | sort -nr  # top values by count
```

Worked example: top 5 IPs in an access log

```
# access.log line format:
# 192.168.1.5 - - [25/May/2026:11:00:01] "GET /api/users HTTP/1.1" 200 1234

awk '{print $1}' access.log |   # extract the IP (1st column)
    sort |                       # sort so identical IPs are adjacent
    uniq -c |                    # count each unique IP
    sort -nr |                   # sort by count descending
    head -5                      # take the top 5
```

That's a complete data pipeline in five lines, with no Python, no dependencies, no parsing libraries – and it runs in milliseconds on a million-line log.

Other tools worth knowing

```
head -n 20 file.txt            # first 20 lines (default 10)
tail -n 20 file.txt            # last 20 lines
tail -f app.log                # live-follow a log file (Ctrl+C to stop)
cut -d, -f1,3 data.csv         # columns 1 and 3 from a CSV
tr ',' '\t' < data.csv         # translate characters (CSV -> TSV)
wc -l file.txt                 # count lines
wc -c file.txt                 # count bytes
xargs                          # turn stdin into command-line arguments
```

Part III — Data Wrangling

8 CSV and TSV with `awk`, `miller`, and `csvkit`

`awk` from the previous section is enough for simple column-oriented work. For headers-aware CSV processing (the kind where you want to say "the `score` column," not "column 4"), two specialized tools step in.

`miller` (`mlr`) – structured CSV processing

```
sudo apt install -y miller      # one-time install
```

Given `users.csv`:

```
user_id,name,role,active,score
1,Alice,Data Engineer,true,0.91
2,Bob,ML Engineer,true,0.87
3,Carol,Analyst,false,0.62
4,Dan,Data Engineer,true,0.78
```

A few mlr one-liners:

```
# Pretty-print the file as a table:
mlr --c2p cat users.csv
# Reads CSV (--c2p = CSV -> pretty)

# Filter rows:
mlr --csv filter '$active == "true" && $score >= 0.8' users.csv

# Project columns:
mlr --csv cut -f name,role,score users.csv

# Sort:
mlr --csv sort -nr score users.csv

# Group-by aggregate -- average score per role:
mlr --csv stats1 -a mean,count -f score -g role users.csv

# Multi-step pipeline:
mlr --csv \
    filter '$active == "true"' then \
    sort -nr score then \
    cut -f name,role,score \
    users.csv
```

`csvkit` – SQL-on-CSV

```
uv tool install csvkit          # if you have uv (recommended)
```

```
# or:  pip install --user csvkit
```

`csvkit` is a suite of Python-based CSV tools:

```
csvlook users.csv                                  # pretty-print
csvcut -c name,score users.csv                      # project columns by name
csvgrep -c role -m "Data Engineer" users.csv        # filter rows
csvstat users.csv                                   # describe each column
csvsql --query "SELECT role, AVG(score) FROM users GROUP BY role" users.csv
```

`csvsql` is killer for ad-hoc analysis: full SQLite SQL, run against any CSV, no schema definition needed.

When to drop into Python

Use bash CSV tools for: file inspection, quick filtering, ad-hoc counts, format conversion. Use `pandas` for: joins, merges, time-series operations, anything vectorized, anything where you'd hate to write the equivalent in shell.

Compare with Python

`mlr --csv filter '$score > 0.8' then sort -nr score` is the shell analog of `df[df.score > 0.8].sort_values('score', ascending=False)`. Both are header-aware, both are fast for medium data. `mlr` keeps you in the shell pipeline so you can pipe to `curl`, `jq`, etc. `pandas` wins as soon as you need joins or numeric heavy lifting.

9 JSON with `jq`

`jq` is to JSON what `awk` is to text: small, composable, ubiquitous. Every modern API speaks JSON; every modern config file is JSON or YAML. `jq` is how you read and transform them from the shell.

Selectors

Given `user.json`:

```
{
  "id": 42,
  "name": "shiru",
  "roles": ["DE", "MLE"],
  "scores": {"q1": 0.91, "q2": 0.88}
}
```

```
jq '.' user.json                # pretty-print
jq '.name' user.json            # "shiru"
jq -r '.name' user.json         # shiru   (-r = raw, unquoted strings)
jq '.roles' user.json           # ["DE","MLE"]
jq '.roles[0]' user.json        # "DE"
jq '.roles[]' user.json         # "DE"\n"MLE"  (emits each as separate value)
```

```
jq '.scores.q1' user.json        # 0.91
```

Filtering arrays

Given `users.json` – an array of objects:

```
[
  {"name": "Alice", "role": "DE",  "score": 0.91},
  {"name": "Bob",   "role": "MLE", "score": 0.87},
  {"name": "Carol", "role": "DA",  "score": 0.62}
]
```

```
# Names of users with score > 0.8:
jq '.[] | select(.score > 0.8) | .name' users.json
# "Alice"
# "Bob"

# Project to a smaller shape:
jq '.[] | {name, role}' users.json

# Group by role -- the "GROUP BY" idiom:
jq 'group_by(.role) | map({role: .[0].role, n: length, avg: (map(.score) |
    add/length)})' users.json
```

The canonical idiom: `curl | jq`

```
# Get the title of every open issue in a public repo:
curl -s https://api.github.com/repos/python/cpython/issues?state=open |
    jq -r '.[] | "#\(.number) \(.title)"'
```

Writing JSON back

```
# Modify a field and write back:
jq '.name = "shiru-new"' user.json > user.tmp.json && mv user.tmp.json user.json

# Build a JSON object from shell variables:
jq -n --arg name "$USER" --argjson n 42 '{user: $name, count: $n}'
# {"user":"shiru","count":42}
```

If you read the PowerShell tutorial

`jq` is the shell-native equivalent of PowerShell's `ConvertFrom-Json` / `ConvertTo-Json` – with one big upgrade: it has no `-Depth` footgun. `jq` preserves arbitrary nesting by default. The trade-off is that `jq`'s filter language is its own DSL, with a learning curve.

10 HTTP and APIs with `curl`

`curl` is the swiss-army HTTP client. It's installed everywhere, supports every protocol you'll need, and pipes straight into `jq`.

Basic GET

```
curl https://api.github.com/repos/python/cpython | jq '.stargazers_count'
# 65000

# -s = silent (no progress bar), -L = follow redirects:
curl -sL https://example.com/data.json | jq '.'
```

Headers, auth, and POST bodies

```
# Bearer token from an environment variable:
curl -s \
    -H "Authorization: Bearer $GITHUB_TOKEN" \
    -H "Accept: application/vnd.github+json" \
    https://api.github.com/repos/python/cpython/pulls?state=open |
    jq -r '.[] | "#\(.number) \(.title)"'

# POST with a JSON body:
curl -s \
    -H "Content-Type: application/json" \
    -d '{"name":"shiru","role":"DE"}' \
    https://httpbin.org/post | jq .

# POST with a body from a file:
curl -s -H "Content-Type: application/json" -d @body.json
    https://api.example.com/items

# Save the response to a file:
curl -sL https://example.com/big.csv -o big.csv

# Useful debugging flags:
curl -v ...         # verbose: show request/response headers
curl -i ...         # include response headers in output
curl -w '%{http_code}\n' -o /dev/null -s ...  # print only the status code
```

A paginated-API example

```
all=$(mktemp)
page=1
while :; do
    chunk=$(curl -s -H "Authorization: Bearer $API_TOKEN" \
        "https://api.example.com/items?page=$page&per_page=100")
    n=$(echo "$chunk" | jq 'length')
    [[ "$n" -eq 0 ]] && break
```

```
    echo "$chunk" | jq '.[]' >> "$all"
    page=$((page + 1))
done

jq -s '.' "$all" > all_items.json      # -s = "slurp" into a single array
rm "$all"
```

Compare with Python

This is the shell version of the `requests.get(..., params={...}).json()` pattern, with `jq` replacing manual JSON traversal. For high-throughput ingestion you'd reach for Python's `httpx` or `aiohttp`; for ad-hoc API exploration, `curl + jq` is hard to beat.

Part IV — Environment and Python

11 Environment Variables, `.bashrc`, and `dotenv`

Setting variables for the current shell

```
AWS_PROFILE=production          # for THIS process only (not children)
export AWS_PROFILE=production   # for this process AND every child it spawns

echo $AWS_PROFILE               # production
printenv AWS_PROFILE            # also production
env | grep AWS                  # every AWS_* in this shell
```

Gotcha

Without `export`, child processes (like the Python script you just ran) cannot see the variable. As a rule: **always** `export` for anything a tool needs to read from the environment.

Persisting variables: `.bashrc`

The file `~/.bashrc` runs every time you open an interactive shell. Add `export` lines there:

```
# Edit with your favorite editor:
nano ~/.bashrc          # or vim, or `code ~/.bashrc`

# Add at the bottom:
export AWS_DEFAULT_REGION=us-east-1
export EDITOR=nano
export PATH="$HOME/.local/bin:$PATH"

# Apply WITHOUT reopening the shell:
source ~/.bashrc
```

Gotcha

`.bashrc` runs for *interactive* shells. `.bash_profile` (or `.profile`) runs for *login* shells. On Ubuntu WSL2, you usually only need `.bashrc`, and the default `.profile` already sources it. If your `export`s aren't taking effect in a new shell, check both files.

The `.env` pattern – credentials without leaks

The standard pattern: store secrets in a per-project `.env` file, `gitignore` it, and load it on demand.

A file `~/projects/etl/.env`:

```
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
AWS_DEFAULT_REGION=us-east-1
MONGO_URI=mongodb+srv://user:pass@cluster.example.net/
```

```
MYSQL_HOST=db.internal
MYSQL_USER=shiru
MYSQL_PASSWORD=...
```

Load it into the current shell:

```
set -a                # automatically export every variable assignment that follows
source .env           # run the file (each FOO=bar becomes an export)
set +a                # turn auto-export back off
```

Now `aws`, `mongosh`, `mysql`, and any Python script you launch will see those credentials. Add `.env` to `.gitignore` and you'll never accidentally commit secrets.

Inspecting and modifying `PATH`

```
echo "$PATH"                        # one long colon-separated string
echo "$PATH" | tr ':' '\n'          # one entry per line, readable

# Add a directory to the front of PATH (so it wins over system tools):
export PATH="$HOME/.local/bin:$PATH"

# Remove a duplicate manually -- it's just string editing.
```

12 Python on Ubuntu

There are three Pythons you'll encounter on Ubuntu:

1. **System Python** (`python3`, installed by the OS, used by apt, dpkg, and many system services). **Do not `pip install` into it.** Breaking it can break Ubuntu itself.
2. **`pyenv`-managed Pythons** – a tool that installs multiple Python versions side-by-side in your home directory. Useful if you need, say, 3.10 for one project and 3.12 for another.
3. **`uv`-managed Pythons** – the modern path. `uv` downloads the exact Python version each project requests, no `pyenv` needed.

The recommended setup for a new WSL2 install: **install `uv`, skip `pyenv`.**

Installing `uv`

```
curl -LsSf https://astral.sh/uv/install.sh | sh

# Add uv to PATH if the installer prompts you (it usually edits ~/.bashrc):
source ~/.bashrc

uv --version
# uv 0.x.y
```

Letting uv manage Python

```
uv python install 3.12          # download and install Python 3.12
uv python list                   # show every Python uv knows about
uv python pin 3.12               # set the version for THIS project (writes
    .python-version)
```

Running a script with no venv ceremony

```
# Run a script that has inline dependency metadata (PEP 723):
uv run --with pandas,boto3 python -c "import pandas; print(pandas.__version__)"
```

For more substantial work, you'll use a project + venv – next section.

13 Virtual Environments with uv and venv

The classic way – python -m venv

```
cd ~/projects/my-app
python3 -m venv .venv              # create
source .venv/bin/activate          # activate (prompt now shows (.venv))
python -m pip install -U pip
python -m pip install pandas boto3 pymongo mysql-connector-python
# work, work, work...
deactivate                          # leave the venv
```

If you read the PowerShell tutorial

On PowerShell you'd run `.\.venv\Scripts\Activate.ps1` – and the first time, you'd hit the execution-policy wall and have to run `Set-ExecutionPolicy` once. On Linux there is no execution-policy: just `source .venv/bin/activate` and you are in.

The modern way – uv

```
cd ~/projects
uv init churn-model              # scaffolds pyproject.toml + .venv +
    .python-version + uv.lock
cd churn-model

uv add pandas boto3 pymongo "mysql-connector-python>=8.3"
uv add --dev pytest ruff

# Run a Python script in the project venv -- no manual activation needed:
uv run python -m churn_model.train --input data/users.csv

# One-off commands:
uv run pytest
uv run ruff check .
```

```
# Make the venv match uv.lock exactly (use this on CI / fresh clones):
uv sync
```

When to activate vs use `uv run`

- **Interactive work, REPLs, debugging:** `source .venv/bin/activate` once, then `python`, `ipython`, `pytest` feel native.
- **Scripts, CI, one-off commands:** `uv run ...` each time. No activation, no risk of forgetting to deactivate.

Legacy: `requirements.txt`

```
uv venv                                          # create .venv
uv pip install -r requirements.txt               # install (fast
    pip-compatible)
uv pip compile requirements.in -o requirements.txt   # resolve & pin
uv pip sync requirements.txt                     # make .venv match EXACTLY
```

Part V — The Data & ML Ecosystem

14 Docker on WSL2

Docker on WSL2 is the easiest Linux Docker setup that exists. Two paths to install it:

Path A: Docker Desktop with the WSL2 backend (recommended)

Install steps (one-time):

1. Download Docker Desktop for Windows from `https://www.docker.com/products/docker-desktop/`.
2. Run the installer; check *Use WSL 2 instead of Hyper-V* when offered.
3. Start Docker Desktop. In Settings → Resources → WSL Integration, enable integration with your Ubuntu distro.

After this, `docker` is available inside Ubuntu and there's no separate Linux Docker daemon to manage. Docker Desktop is free for individuals and small businesses; check the license if you're on a larger team.

Path B: Rootless Docker inside Ubuntu

If you'd rather have no Windows-side service, install Docker's `rootless` variant entirely inside WSL2. The official one-liner is at `https://docs.docker.com/engine/security/rootless/`. The trade-offs: a slightly less seamless experience (you start the daemon manually), but full isolation from Windows.

Daily commands

```
docker --version
docker ps                          # running containers
docker ps -a                       # all containers (including stopped)
docker images                      # local images
docker pull postgres:16            # download an image
docker run --rm -it ubuntu:24.04 bash   # interactive shell in a fresh container
                                    # --rm = remove on exit
                                    # -it = interactive + TTY

# Background ("detached") with a port mapping and a name:
docker run -d --name pg \
    -e POSTGRES_PASSWORD=secret \
    -p 5432:5432 \
    postgres:16

docker logs pg                     # see its stdout
docker exec -it pg psql -U postgres # run psql inside the container
docker stop pg && docker rm pg     # tear down
```

`docker compose` for multi-container setups

A file `docker-compose.yml`:

```
services:
  pg:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
  mongo:
    image: mongo:7
    ports:
      - "27017:27017"

volumes:
  pgdata:
```

```
docker compose up -d        # start everything in background
docker compose ps           # show status
docker compose logs -f pg   # follow pg logs
docker compose down         # stop and remove
docker compose down -v      # also drop the named volumes
```

Volumes and bind mounts

```
# Mount your current working directory at /app inside the container:
docker run --rm -v "$PWD":/app -w /app python:3.12 python script.py
# -v HOST:CONTAINER  -- bind mount
# -w /app            -- working directory inside the container
```

Building your own image

A `Dockerfile`:

```
FROM python:3.12-slim

WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev
COPY . .
CMD ["uv", "run", "python", "-m", "myapp"]
```

```
docker build -t myapp:dev .
docker run --rm -p 8000:8000 myapp:dev
```

15 CUDA / GPU Passthrough for ML

WSL2 supports NVIDIA GPU passthrough out of the box, which means your PyTorch and TensorFlow installs inside Ubuntu can use your Windows GPU for training. The single most important rule: **do not install a Linux NVIDIA driver inside Ubuntu.** The GPU is shared from Windows; the only driver that should exist is the Windows one.

Setup checklist

1. Install the latest NVIDIA **Windows** driver from `https://www.nvidia.com/Download/index.aspx`. As of 2026, any driver $\geq$ 535 supports WSL2 CUDA.
2. Inside WSL2 Ubuntu, confirm the GPU is visible:

```
nvidia-smi
# +-------------------------------------+
# | NVIDIA-SMI 545.29 ...               |
# | GPU  Name         ...  Memory ...   |
# | 0    NVIDIA RTX   ...   8192MiB     |
# +-------------------------------------+
```

3. Install PyTorch with CUDA inside your project's venv:

```
cd ~/projects/my-model
uv add torch torchvision --index-url https://download.pytorch.org/whl/cu121
# (Pick the cu1XX matching your CUDA toolkit; cu121 = CUDA 12.1, current at time
    of writing.)
```

4. Sanity-check inside Python:

```
import torch
print(torch.cuda.is_available())   # True
print(torch.cuda.device_count())   # 1
print(torch.cuda.get_device_name(0))
```

Tuning WSL2 memory

By default, WSL2 takes up to 50% of your Windows RAM. For ML work you usually want more (or less, if you're hitting Windows OOM). Create `C:\Users\shiru\.wslconfig` (on Windows, not inside WSL):

```
[wsl2]
memory=24GB
processors=8
swap=8GB
```

Then `wsl --shutdown` from PowerShell to make it take effect.

Gotcha

`nvidia-smi` working but `torch.cuda.is_available()` returning `False` usually means you installed the *CPU* build of PyTorch. Reinstall with the explicit `--index-url` for the right CUDA version.

16 AWS, MongoDB, and MySQL Clients in 30 Lines

AWS CLI v2

```
curl -sL "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip
sudo ./aws/install
rm -rf aws awscliv2.zip

aws --version
# aws-cli/2.x.x ...

# Configure interactively (writes ~/.aws/credentials and ~/.aws/config):
aws configure

# Or use env vars (set in .env, then `set -a; source .env; set +a`):
aws s3 ls s3://my-bucket/
aws s3 cp data.csv s3://my-bucket/raw/data.csv
aws s3 sync ./local-dir s3://my-bucket/dir/
```

MongoDB shell

```
# Install mongosh from MongoDB's apt repo (one-time):
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | sudo gpg --dearmor
    -o /usr/share/keyrings/mongodb.gpg
echo "deb [signed-by=/usr/share/keyrings/mongodb.gpg]
    https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" | sudo
    tee /etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt update && sudo apt install -y mongodb-mongosh

# Connect:
mongosh "$MONGO_URI"

# Or, run a one-off query and exit:
mongosh "$MONGO_URI" --eval 'db.users.countDocuments({active: true})'
```

MySQL client

```
sudo apt install -y mysql-client

mysql -h "$MYSQL_HOST" -u "$MYSQL_USER" -p"$MYSQL_PASSWORD" mydb
# (Note: -p PASSWORD with no space; or just -p to be prompted.)
```

```
# Run a one-off query:
mysql -h "$MYSQL_HOST" -u "$MYSQL_USER" -p"$MYSQL_PASSWORD" -e "SELECT COUNT(*)
    FROM users" mydb

# Dump and restore:
mysqldump -h "$MYSQL_HOST" -u "$MYSQL_USER" -p"$MYSQL_PASSWORD" mydb > mydb.sql
mysql    -h "$MYSQL_HOST" -u "$MYSQL_USER" -p"$MYSQL_PASSWORD" mydb < mydb.sql
```

Part VI — Scripting and a Real ETL

17 Bash Scripting Basics

A *script* is just a text file of bash commands with a shebang line and the execute bit set:

```
#!/usr/bin/env bash
echo "hello, $1"
```

```
chmod +x hello.sh
./hello.sh shiru
# hello, shiru
```

The discipline four: `set -euo pipefail`

Put this at the top of every non-trivial bash script:

```
#!/usr/bin/env bash
set -euo pipefail

# Optional during debugging:
# set -x   # echo every command before running it
```

What each flag does:

Flag	Effect
`-e`	Exit immediately on any failed command
`-u`	Treat unset variables as an error (catches typos)
`-o pipefail`	A pipeline fails if ANY command in it fails (not just the last)
`-x`	Print each command before running – only for debugging

Without these, bash's defaults are alarmingly permissive: a missing variable becomes an empty string (so `rm -rf "$DIR/"` becomes `rm -rf /` if `$DIR` is unset), and a failed pipe stage is silently ignored.

Functions, arguments, return codes

```
#!/usr/bin/env bash
set -euo pipefail

log() {
    local msg="$1"
    printf '%s %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$msg" >&2
}

ensure_dir() {
    local d="$1"
    if [[ ! -d "$d" ]]; then
```

```
        mkdir -p "$d"
        log "created $d"
    fi
}

log "starting"
ensure_dir /tmp/work
log "done"
```

Conditionals: `[[ ... ]]`

```
if [[ -f users.csv ]]; then
    echo "file exists"
fi

if [[ -z "$VAR" ]]; then
    echo "VAR is empty or unset"
elif [[ "$VAR" == "prod" ]]; then
    echo "production mode"
else
    echo "VAR = $VAR"
fi

# Common tests:
# -f file     -- file exists
# -d dir      -- directory exists
# -z str      -- string is empty
# -n str      -- string is non-empty
# str1 == str2  -- equality (string)
# str1 =~ regex -- regex match
# n -eq m, -lt, -gt -- numeric comparisons
# cond1 && cond2, cond1 || cond2 -- combine
```

Gotcha

The older single-bracket form `[ ... ]` (a POSIX-portable test) works but has more quoting pitfalls. In bash scripts, **always use `[[ ... ]]`** – it's safer and supports `=~` (regex) and `&&`/`||` inside.

Loops

```
# Over files (preferred -- handles spaces correctly):
for f in *.csv; do
    echo "processing $f"
done

# Over an explicit list:
for env in dev stage prod; do
    deploy "$env"
```

```
done

# Over lines of a file (the safe form -- no word-splitting bugs):
while IFS= read -r line; do
    echo "got: $line"
done < input.txt

# C-style counted loop:
for ((i=0; i<10; i++)); do
    echo "$i"
done
```

`trap` for cleanup

```
#!/usr/bin/env bash
set -euo pipefail

tmpdir=$(mktemp -d)
trap 'rm -rf "$tmpdir"' EXIT      # ALWAYS run on script exit, success or failure

# ... work in $tmpdir ...
```

Parsing CLI arguments

```
#!/usr/bin/env bash
set -euo pipefail

input=""
output=""
verbose=false

while [[ $# -gt 0 ]]; do
    case "$1" in
        -i|--input)   input="$2";  shift 2 ;;
        -o|--output)  output="$2"; shift 2 ;;
        -v|--verbose) verbose=true; shift ;;
        -h|--help)    echo "usage: $0 -i INPUT -o OUTPUT"; exit 0 ;;
        *)            echo "unknown arg: $1" >&2; exit 2 ;;
    esac
done

[[ -z "$input"  ]] && { echo "missing --input"  >&2; exit 2; }
[[ -z "$output" ]] && { echo "missing --output" >&2; exit 2; }

echo "input=$input output=$output verbose=$verbose"
```

18 Putting It Together — A Mini ETL Walkthrough

Goal: read a CSV of users, enrich each row by hitting a REST API, run a Python transformation in a uv-managed venv to compute a per-user score, write the result as JSON, and log progress to a daily log file.

Project layout

```
~/projects/enrich/
    pyproject.toml          # managed by uv
    uv.lock
    .venv/                   # created by `uv venv`
    .env                     # secrets (gitignored)
    data/
        users.csv            # input
    enrich.sh                # orchestrator (bash)
    score.py                 # transformation (Python)
    logs/
        enrich-2026-05-25.log
```

The Python transformation (`score.py`)

```
"""Read JSON lines from stdin; emit scored JSON lines on stdout."""
import json, sys

for line in sys.stdin:
    rec = json.loads(line)
    base = float(rec.get("score", 0))
    boost = 0.1 if rec.get("verified") else 0.0
    rec["final_score"] = round(base + boost, 4)
    sys.stdout.write(json.dumps(rec) + "\n")
```

The orchestrator (`enrich.sh`)

```
#!/usr/bin/env bash
set -euo pipefail

INPUT="${INPUT:-data/users.csv}"
OUTPUT="${OUTPUT:-data/users_scored.json}"

# Load .env if present (export every variable in it):
if [[ -f .env ]]; then
    set -a; source .env; set +a
fi

# Set up daily log file:
mkdir -p logs
LOG="logs/enrich-$(date +%F).log"
```

```
log() {
    printf '%s %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$*" | tee -a "$LOG"
}

log "Reading $INPUT"
n_users=$(($(wc -l < "$INPUT") - 1))   # subtract header
log "Enriching $n_users users via API"

# Convert CSV rows to JSON, hit the API for each user_id, merge response in,
# then pipe everything to score.py via uv run, and write a JSON array out.
tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT

mlr --c2j cat "$INPUT" | jq -c '.[]' | while read -r row; do
    user_id=$(jq -r '.user_id' <<< "$row")
    profile=$(curl -fsS \
        -H "Authorization: Bearer $API_TOKEN" \
        "https://api.example.com/users/$user_id" || echo '{}')
    jq -c --argjson p "$profile" '. + {verified: ($p.verified // false)}' <<<
    "$row"
done > "$tmp"

log "Scoring $(wc -l < "$tmp") records via Python (uv run)"
uv run python score.py < "$tmp" | jq -s '.' > "$OUTPUT"

log "Wrote $(jq 'length' "$OUTPUT") records to $OUTPUT"
log "Done"
```

Run it

```
chmod +x enrich.sh
./enrich.sh
# or with overrides:
INPUT=data/sample.csv OUTPUT=/tmp/out.json ./enrich.sh
```

What this script demonstrates, in one place:

- `set -euo pipefail` discipline.
- Environment defaults with parameter expansion (`${INPUT:-data/users.csv}`).
- `.env`-based secrets loading.
- Per-day log file with a small `log()` helper.
- Conversion between CSV and JSON with `mlr --c2j` and `jq`.
- API enrichment with `curl` (and `-fsS` – fail on errors, silent progress, but show real errors).
- Hand-off to Python via `uv run python score.py`, streaming JSON lines through stdin/stdout.
- Cleanup via `trap '...' EXIT`.

Part VII — Reference

19 Common Pitfalls

The list of bash footguns every newcomer hits exactly once. Read it once, recognize them next time.

`FOO = bar` **(spaces) is not assignment.** Bash parses it as "run the command `FOO` with arguments `=` and `bar`." Always `FOO=bar`, no spaces.

Word-splitting unquoted variables. `rm $file` breaks if `$file` contains spaces or expands to a glob. Always `rm "$file"` unless you specifically want splitting.

`cd $(dirname "$0")` **vs** `cd "$(dirname "$0")".` Without the outer quotes, a script in a path with spaces breaks. Always wrap the substitution in quotes too.

Line endings. If you edit a script on Windows in a tool that writes `\r\n`, the shebang becomes `#!/usr/bin/env bash\r` and you'll see "no such file or directory" when you run it. Use VS Code's WSL-side editor, or `dos2unix script.sh`.

`/mnt/c/` **is slow.** Keep code under `~/`. Anything that touches many files (git, pip, npm) is dramatically faster on the Linux filesystem.

`apt install` **for Python packages.** Don't. Use `uv add` or `pip install` inside a venv. Touching system Python with `sudo pip` can break Ubuntu.

`sudo` **and** `PATH`**.** `sudo` resets `PATH` to a minimal set, so `sudo my-tool` may fail with "command not found" even though `my-tool` is on your `PATH`. Use the absolute path: `sudo /home/shiru/.local/bin/my-t`

Editing `.bashrc` **and forgetting to** `source`**.** Your changes don't take effect until you start a new shell or run `source ~/.bashrc`.

`rm -rf $VAR/`**.** If `$VAR` is unset or empty, this is `rm -rf /`. The `set -u` flag prevents the unset case; quoting (`"$VAR/"`) doesn't help. The safest form is to check first: `[[ -n "$VAR" && -d "$VAR" ]] && rm -rf "$VAR"`.

Pipes hide failures. `cmd1 | cmd2` only reports `cmd2`'s exit code. Use `set -o pipefail` so a failing `cmd1` fails the pipe.

Single vs double quotes inside `jq` **expressions.** Use single quotes around the whole `jq` program so bash doesn't expand `$.field` – inside the program, `.field` is the jq syntax, not a shell variable.

`nvidia-smi` **works but PyTorch can't see the GPU.** You installed the CPU build. Reinstall PyTorch with the `--index-url` matching your CUDA version (Section 14's neighbor).

20 Quick Reference Card

The three-column lookup: a concept you know from Python, the bash command, and – if you read the PowerShell tutorial – the equivalent there.

Python / Concept	Bash	PowerShell
`os.getcwd()`	`pwd`	`Get-Location`
`os.chdir('/p')`	`cd /p`	`Set-Location`
`os.listdir()`	`ls`	`Get-ChildItem`
`os.listdir` (with hidden)	`ls -la`	`Get-ChildItem -Force`
`glob.glob('**/*.csv')`	`ls **/*.csv` (globstar)	`Get-ChildItem -Recurse -Filter *.csv`
`open(p).read()`	`cat p`	`Get-Content p`
`open(p).readlines()[:10]`	`head -n 10 p`	`Get-Content p -TotalCount 10`
`collections.deque(open(p),10)`	`tail -n 10 p`	`Get-Content p -Tail 10`
`tail -f`	`tail -f p`	`Get-Content p -Wait -Tail 0`
`len(open(p).readlines())`	`wc -l p`	`(Get-Content p | Measure-Object -Line).Lines`
`re.findall(pat, text)`	`grep -E pat p`	`Select-String`
`shutil.which('python')`	`which python`	`(Get-Command python).Source`
`os.makedirs(p)`	`mkdir -p p`	`New-Item -ItemType Directory -Force p`
`open(p,'w').close()`	`touch p`	`New-Item -ItemType File p`
`shutil.copy(s,d)`	`cp s d`	`Copy-Item s d`
`shutil.copytree(s,d)`	`cp -r s d`	`Copy-Item -Recurse s d`
`shutil.move(s,d)`	`mv s d`	`Move-Item s d`
`os.remove(p)`	`rm p`	`Remove-Item p`
`shutil.rmtree(p)`	`rm -rf p`	`Remove-Item -Recurse -Force p`
`os.symlink(t,l)`	`ln -s t l`	`New-Item -ItemType SymbolicLink ...`
`os.environ['V']`	`$V` or `${V}`	`$env:V`
`os.environ['V'] = 'x'`	`export V=x`	`$env:V = 'x'`
`json.load(open(p))`	`jq . p`	`Get-Content p | ConvertFrom-Json`
`csv.DictReader(open(p))`	`mlr --csv cat p`	`Import-Csv p`
`requests.get(u).json()`	`curl -s u | jq .`	`Invoke-RestMethod u`
`requests.post(u, json=b)`	`curl -d @b.json -H Content-Type:application/json u`	`Invoke-RestMethod -Method Post -Body $b u`
`subprocess.run([...])`	`cmd $arg1 $arg2`	(run cmdlet directly)
`cmd1 && cmd2`	`cmd1 && cmd2`	`cmd1 && cmd2` (PS 7+)
`a = subprocess.check_output(c)`	`a=$(c)`	`$a = (c)`

Last advice. When you're stuck, the answer almost always starts with one of three commands: `man cmd` (the manual page), `cmd --help` (the short help), or `type cmd` (is it a built-in, an alias, or a binary on `PATH`?). Bash rewards a small set of well-learned tools over a large set of half-known ones – pick five from this tutorial (`grep`, `awk`, `jq`, `curl`, `uv`) and use them every day for two weeks. You'll be fluent.